Critical Thinking Module 4: Operating System First-Fit Memory Allocation

Alexander Ricciardi

Colorado State University Global

CSC507: Foundations of OSs

Professor: Dr. Joseph Issa

Jun 7, 2026

Critical Thinking Module 4: Operating System First-Fit Memory Allocation

Operating Systems (OSs) use a memory allocator to manage the Random Access Memory (RAM) in computer systems. The memory allocator's main job is to track where processes are loaded into RAM. The main challenge is to manage processes that have different sizes and need to be loaded into RAM over time. Moreover, an OS memory allocator's duties involve deciding how to track free memory space, protect active processes, and keep memory usable instead of letting it break into scattered fragments. Additionally, the memory allocator must keep track of addresses, coordinate access, and handle memory fragmentation, which makes the memory total free space look sufficient even though there is no single contiguous block that is large enough to meet the memory needs of a large process (CSU Global, n.d.).

This paper examines the First-Fit algorithm using a Python program. The Python script code logic simulates a dynamic memory allocation system that an OS may use. The program also implements different memory block sizes, process sizes, and request patterns to test the simulated memory allocation system. To match the First-Fit algorithm functionality, the program stores free memory as a list of contiguous holes, allocates each process to the first hole large enough to hold it, and records the outcome of each decision; see Figure 1 and Figure 2. Additionally, the program runs three scenarios to test first-fit, best-fit, and side-by-side comparison functionality, exporting to outputs to CSV files; see Figure 3. This allows the outputs to be examined. The examination of the outputs showed that First-Fit algorithm is more efficient and practical as it is simpler and inspects fewer holes. On the other hand, the Best-Fit algorithm is more suited to situations where memory is restricted and preserving larger holes for later searches matters more than search cost.

The First-Fit Algorithm

The First-Fit algorithm is a sequential memory allocation approach that minimizes overhead created by memory search; it implements a simple rule that assigns a first-come process to the very first available block of free memory that is large enough to accommodate it (Wilson et al., 1995). First, it scans a free-hole list from the lowest address to the highest address and stops at the first large enough hole to fit the process memory requirement. If the hole is larger than the process, the OS places the process at the front of that hole and leaves the remainder available as a smaller free block. If no hole is large enough, the allocation fails. This allocation implementation is simple, but it does not solve the larger fragmentation problem created by dynamic partitioning. This flexibility provided by the dynamic approach is valuable, but it also partitions memory free space into pieces that are too small to accommodate large processes' memory needs, even when the total free memory is sufficient to accommodate these processes' memory needs. This means that the memory allocator efficiency needs to be judged not only by how quickly it answers one request, but also by the shape of the memory that it leaves free for the next request. In other words, the First-Fit algorithm favors low search overhead over fully scanning the free hole list. The program associated with the project implements that exact functionality, see Figure 2. For each process, it walks the current free hole list, records the placement address when a big enough hole is found, subtracts the process size from that hole, and keeps any remainder in the free list. When no large enough hole is found, it records a failed allocation and identifies whether the failure was caused by external fragmentation rather than by an absolute lack of free memory.

Figure 1

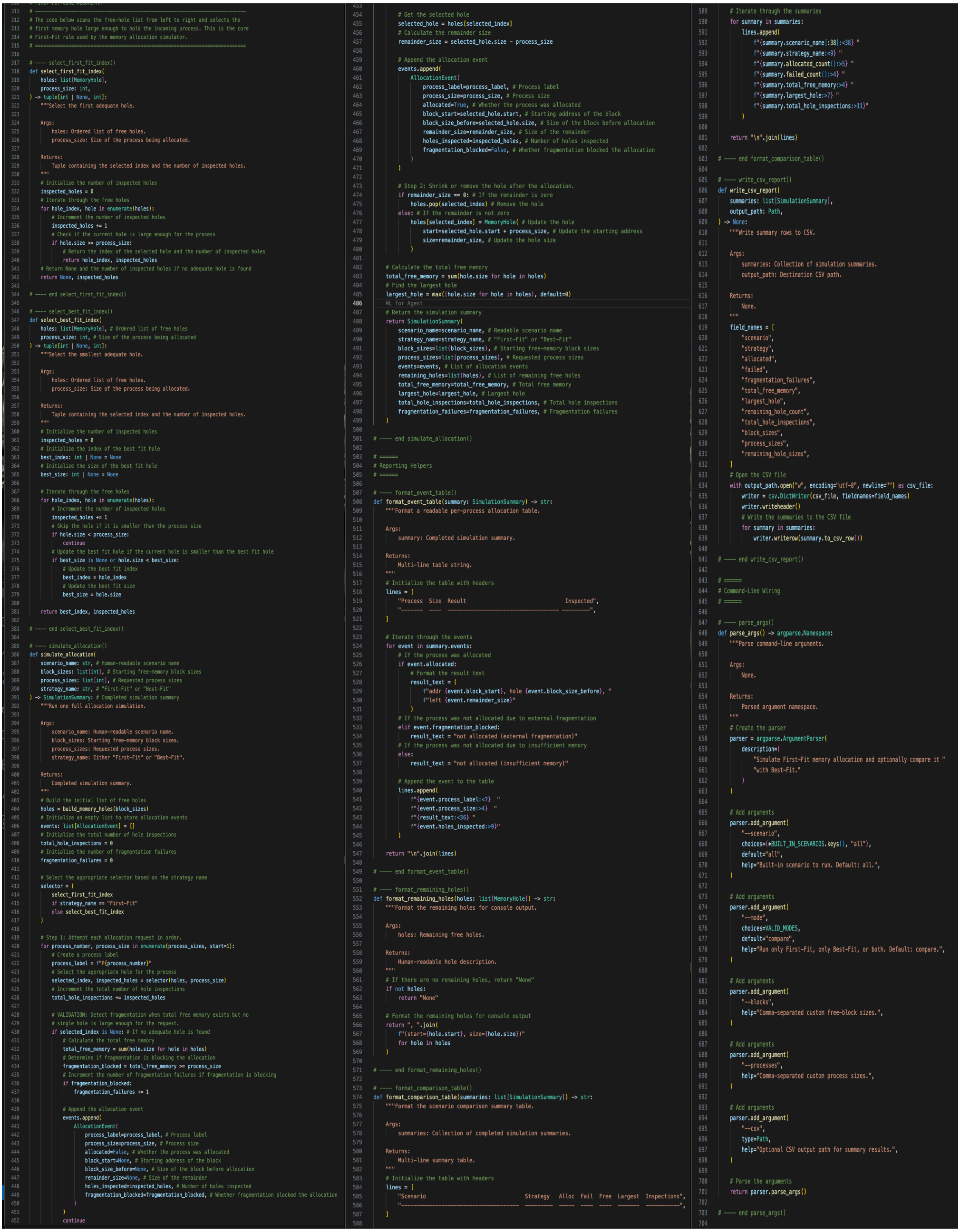
Program setup

```
# File: first_fit_memory_allocation.py
# Author: Alexander Ricciardi
# Date: 2026-06-07
# Course: CSC507
# Professor: Dr. Joseph Issa
# Term: Spring C 2026

"""The Program simulates contiguous memory allocation with the First-Fit Algorithm.

This standalone Python script simulates free memory as an ordered list of holes.
It simulates three distinct memory allocation scenarios or a custom set of memory
blocks and process sizes. The script can run in one of three modes:
compare, first-fit, or best-fit. First-Fit and Best-Fit can be compared side by side
and summary metrics can be exported to CSV.
"""

from __future__ import annotations

import argparse
import csv
from dataclasses import dataclass
from pathlib import Path


# ======
# Constants
# ======

DEFAULT_CSV_PATH = Path("first_fit_results.csv")
VALID_MODES = ("compare", "first-fit", "best-fit")


# ======
# Data Models / Classes
# ======

# ---- class MemoryHole
@dataclass(frozen=True)
class MemoryHole:
    """Represent one free contiguous memory region.

    Args:
        start: Starting address of the free hole.
        size: Number of addressable units in the free hole.

    Returns:
        None.
    """

    start: int
    size: int

# ---- end class MemoryHole

# ---- class AllocationEvent
@dataclass(frozen=True)
class AllocationEvent:
    """Store the result of one process-allocation attempt.

    Args:
        process_label: Process label such as P1 or P2.
        process_size: Requested memory size.
        allocated: Whether the process was placed into memory.
        block_start: Starting address of the selected hole.
        block_size_before: Size of the selected hole before placement.
        remainder_size: Remaining free size after placement.
        holes_inspected: Number of free holes inspected for this request.
        fragmentation_blocked: Whether failure occurred despite enough total
            free memory being available across smaller holes.

    Returns:
        None.
    """

    process_label: str # Label for the process
    process_size: int # Size of the process
    allocated: bool # Whether the process was allocated
    block_start: int | None # Starting address of the selected hole
    block_size_before: int | None # Size of the selected hole before placement
    remainder_size: int | None # Remaining free size after placement
    holes_inspected: int # Number of free holes inspected for this request
    fragmentation_blocked: bool # Whether failure occurred despite enough total

# ---- end class AllocationEvent

# ---- class SimulationSummary
@dataclass(frozen=True)
class SimulationSummary:
    """Store one finished allocation simulation.

    Args:
        scenario_name: Human-readable scenario name.
        strategy_name: Allocation strategy name.
        block_sizes: Starting free-memory block sizes.
        process_sizes: Requested process sizes.
        events: Ordered allocation events for all processes.
        remaining_holes: Free holes remaining after the simulation.
        total_free_memory: Sum of all remaining free memory.
        largest_hole: Size of the largest remaining hole.
        total_hole_inspections: Total free-hole checks performed.
        fragmentation_failures: Count of failures caused by fragmentation.

    Returns:
        None.
    """

    scenario_name: str # Human-readable scenario name.
    strategy_name: str # Allocation strategy name.
    block_sizes: list[int] # Starting free-memory block sizes.
    process_sizes: list[int] # Requested process sizes.
    events: list[AllocationEvent] # Ordered allocation events for all processes.
    remaining_holes: list[MemoryHole] # Free holes remaining after the simulation.
    total_free_memory: int # Sum of all remaining free memory.
    largest_hole: int # Size of the largest remaining hole.
    total_hole_inspections: int # Total free-hole checks performed.
    fragmentation_failures: int # Count of failures caused by fragmentation.

    # ---- allocated_count()
    def allocated_count(self) -> int:
        """Count successfully allocated processes.

        Args:
            None.

        Returns:
            Number of successful allocations.
        """
        return sum(1 for event in self.events if event.allocated)

    # ---- end allocated_count()

    # ---- failed_count()
    def failed_count(self) -> int:
        """Count failed process allocations.

        Args:
            None.

        Returns:
            Number of failed allocations.
        """
        return sum(1 for event in self.events if not event.allocated)

    # ---- end failed_count()

    # ---- to_csv_row()
    def to_csv_row(self) -> dict[str, str | int]:
        """Convert the summary to a CSV-ready dictionary.

        Args:
            None.

        Returns:
            Dictionary containing CSV columns.
        """
        return {
            "scenario": self.scenario_name,
            "strategy": self.strategy_name,
            "allocated": self.allocated_count(),
            "failed": self.failed_count(),
            "fragmentation_failures": self.fragmentation_failures,
            "total_free_memory": self.total_free_memory,
            "largest_hole": self.largest_hole,
            "remaining_hole_count": len(self.remaining_holes),
            "total_hole_inspections": self.total_hole_inspections,
            "block_sizes": ",".join(str(size) for size in self.block_sizes),
            "process_sizes": ",".join(str(size) for size in self.process_sizes),
            "remaining_hole_sizes": ",".join(
                str(hole.size) for hole in self.remaining_holes
            ),
        }

    # ---- end to_csv_row()

# ---- end class SimulationSummary

# ---- class ScenarioDefinition
@dataclass(frozen=True)
class ScenarioDefinition:
    """Store one named scenario used by the simulator.

    Args:
        name: Human-readable scenario name.
        block_sizes: Starting free-memory block sizes.
        process_sizes: Requested process sizes.
        note: Short explanation of why the scenario is useful.

    Returns:
        None.
    """

    name: str # Human-readable scenario name.
    block_sizes: list[int] # Starting free-memory block sizes.
    process_sizes: list[int] # Requested process sizes.
    note: str # Short explanation of why the scenario is useful.

# ---- end class ScenarioDefinition

# ======
# Scenario Catalog
# ======

# ---- Dictionary of built-in scenarios
BUILT_IN_SCENARIOS: dict[str, ScenarioDefinition] = {
    "balanced": ScenarioDefinition(
        name="Balanced mixed-size requests", # Name of the scenario
        block_sizes=[100, 500, 200, 300, 600], # Free memory block sizes
        process_sizes=[212, 417, 112, 426], # Process sizes to be allocated
        note=(
            "Shows how a later large request can fail under First-Fit even when "
            "the system still has enough total free memory."
        ),
    ),
    "clustered": ScenarioDefinition(
        name="Clustered small and medium requests", # Name of the scenario
        block_sizes=[150, 350, 120, 500, 275], # Free memory block sizes
        process_sizes=[115, 90, 130, 260, 50, 200], # Process sizes to be allocated
        note=(
            "Shows a workload where both strategies allocate every process, but "
            "Best-Fit spends more search effort and leaves very small fragments."
        ),
    ),
    "pressure": ScenarioDefinition(
        name="High fragmentation pressure", # Name of the scenario
        block_sizes=[400, 180, 220, 260, 300], # Free memory block sizes
        process_sizes=[175, 205, 90, 240, 60, 180, 110], # Process sizes to be allocated
        note=(
            "Pushes several medium and small requests through the allocator to "
            "compare final hole layout under sustained pressure."
        ),
    ),
}
# ---- End dictionary of built-in scenarios

# ======
# Parsing and Validation Helpers
# ======

# ---- parse_positive_csv()
def parse_positive_csv(raw_value: str) -> list[int]:
    """Parse a comma-separated list of positive integers.

    Args:
        raw_value: Raw comma-separated string.

    Returns:
        Parsed list of positive integers.

    Raises:
        ValueError: If the input is empty or contains invalid numbers.
    """
    # Initialize an empty list to store parsed values
    parsed_values: list[int] = []

    # Split the raw value by commas and iterate through each piece
    for piece in raw_value.split(","):
        # Remove leading/trailing whitespace from each piece
        cleaned_piece = piece.strip()
        # Skip empty pieces
        if not cleaned_piece:
            continue

        # Convert the cleaned piece to an integer
        parsed_value = int(cleaned_piece)

        # Check if the parsed value is positive
        if parsed_value <= 0:
            # Raise ValueError if the parsed value is not positive
            raise ValueError("all sizes must be greater than zero")

        # Append the parsed value to the list
        parsed_values.append(parsed_value)

    # Check if the list is empty
    if not parsed_values:
        # Raise ValueError if the list is empty
        raise ValueError("at least one positive integer is required")
    # Return the list of parsed values
    return parsed_values

# ---- end parse_positive_csv()

# ---- build_memory_holes()
def build_memory_holes(block_sizes: list[int]) -> list[MemoryHole]:
    """Build the starting free-hole list from block sizes.

    Args:
        block_sizes: Sizes of the starting free holes.

    Returns:
        Ordered list of free memory holes.
    """
    # Initialize an empty list to store free holes
    holes: list[MemoryHole] = []
    # Initialize the starting address of the next hole
    next_start = 0

    # Iterate through the block sizes
    for block_size in block_sizes:
        # Create a new hole with the current block size
        holes.append(MemoryHole(start=next_start, size=block_size))
        # Update the starting address for the next hole
        next_start += block_size

    return holes

# ---- end build_memory_holes()
```

Note: The image shows Python code used to set up the program; the code uses data classes to store memory holes, allocation events, simulation summaries, and scenario definitions.

Figure 2

Python Code First-Fit Core Logic



```python
# ============================================================================
# The code below scans the free-hole list from left to right and selects the
# first memory hole large enough to hold the incoming process. This is the core
# First-Fit rule used by the memory allocation simulator.
# ============================================================================

# ---- select_first_fit_index()
def select_first_fit_index(
    holes: list[MemoryHole],
    process_size: int,
) -> tuple[int | None, int]:
    """Select the first adequate hole.

    Args:
        holes: Ordered list of free holes.
        process_size: Size of the process being allocated.

    Returns:
        Tuple containing the selected index and the number of inspected holes.
    """
    # Initialize the number of inspected holes
    inspected_holes = 0
    # Iterate through the free holes
    for hole_index, hole in enumerate(holes):
        # Increment the number of inspected holes
        inspected_holes += 1
        # Check if the current hole is large enough for the process
        if hole.size >= process_size:
            # Return the index of the selected hole and the number of inspected holes
            return hole_index, inspected_holes
    # Return None and the number of inspected holes if no adequate hole is found
    return None, inspected_holes

# ---- end select_first_fit_index()

# ---- select_best_fit_index()
def select_best_fit_index(
    holes: list[MemoryHole], # Ordered list of free holes
    process_size: int, # Size of the process being allocated
) -> tuple[int | None, int]:
    """Select the smallest adequate hole.

    Args:
        holes: Ordered list of free holes.
        process_size: Size of the process being allocated.

    Returns:
        Tuple containing the selected index and the number of inspected holes.
    """
    # Initialize the number of inspected holes
    inspected_holes = 0
    # Initialize the index of the best fit hole
    best_index: int | None = None
    # Initialize the size of the best fit hole
    best_size: int | None = None

    # Iterate through the free holes
    for hole_index, hole in enumerate(holes):
        # Increment the number of inspected holes
        inspected_holes += 1
        # Skip the hole if it is smaller than the process size
        if hole.size < process_size:
            continue
        # Update the best fit hole if the current hole is smaller than the best fit hole
        if best_size is None or hole.size < best_size:
            # Update the best fit index
            best_index = hole_index
            # Update the best fit size
            best_size = hole.size

    return best_index, inspected_holes

# ---- end select_best_fit_index()

# ---- simulate_allocation()
def simulate_allocation(
    scenario_name: str, # Human-readable scenario name
    block_sizes: list[int], # Starting free-memory block sizes
    process_sizes: list[int], # Requested process sizes
    strategy_name: str, # "First-Fit" or "Best-Fit"
) -> SimulationSummary: # Completed simulation summary
    """Run one full allocation simulation.

    Args:
        scenario_name: Human-readable scenario name.
        block_sizes: Starting free-memory block sizes.
        process_sizes: Requested process sizes.
        strategy_name: Either "First-Fit" or "Best-Fit".

    Returns:
        Completed simulation summary.
    """
    # Build the initial list of free holes
    holes = build_memory_holes(block_sizes)
    # Initialize an empty list to store allocation events
    events: list[AllocationEvent] = []
    # Initialize the total number of hole inspections
    total_hole_inspections = 0
    # Initialize the number of fragmentation failures
    fragmentation_failures = 0

    # Select the appropriate selector based on the strategy name
    selector = (
        select_first_fit_index
        if strategy_name == "First-Fit"
        else select_best_fit_index
    )

    # Step 1: Attempt each allocation request in order.
    for process_number, process_size in enumerate(process_sizes, start=1):
        # Create a process label
        process_label = f"P{process_number}"
        # Select the appropriate hole for the process
        selected_index, inspected_holes = selector(holes, process_size)
        # Increment the total number of hole inspections
        total_hole_inspections += inspected_holes

        # VALIDATION: Detect fragmentation when total free memory exists but no
        # single hole is large enough for the request.
        if selected_index is None: # If no adequate hole is found
            # Calculate the total free memory
            total_free_memory = sum(hole.size for hole in holes)
            # Determine if fragmentation is blocking the allocation
            fragmentation_blocked = total_free_memory >= process_size
            # Increment the number of fragmentation failures if fragmentation is blocking
            if fragmentation_blocked:
                fragmentation_failures += 1

            # Append the allocation event
            events.append(
                AllocationEvent(
                    process_label=process_label, # Process label
                    process_size=process_size, # Process size
                    allocated=False, # Whether the process was allocated
                    block_start=None, # Starting address of the block
                    block_size_before=None, # Size of the block before allocation
                    remainder_size=None, # Size of the remainder
                    holes_inspected=inspected_holes, # Number of holes inspected
                    fragmentation_blocked=fragmentation_blocked, # Whether fragmentation blocked the allocation
                )
            )
            continue

        # Get the selected hole
        selected_hole = holes[selected_index]
        # Calculate the remainder size
        remainder_size = selected_hole.size - process_size

        # Append the allocation event
        events.append(
            AllocationEvent(
                process_label=process_label, # Process label
                process_size=process_size, # Process size
                allocated=True, # Whether the process was allocated
                block_start=selected_hole.start, # Starting address of the block
                block_size_before=selected_hole.size, # Size of the block before allocation
                remainder_size=remainder_size, # Size of the remainder
                holes_inspected=inspected_holes, # Number of holes inspected
                fragmentation_blocked=False, # Whether fragmentation blocked the allocation
            )
        )

        # Step 2: Shrink or remove the hole after the allocation.
        if remainder_size == 0: # If the remainder is zero
            holes.pop(selected_index) # Remove the hole
        else: # If the remainder is not zero
            holes[selected_index] = MemoryHole( # Update the hole
                start=selected_hole.start + process_size, # Update the starting address
                size=remainder_size, # Update the hole size
            )

    # Calculate the total free memory
    total_free_memory = sum(hole.size for hole in holes)
    # Find the largest hole
    largest_hole = max((hole.size for hole in holes), default=0)
    ⌘L for Agent
    # Return the simulation summary
    return SimulationSummary(
        scenario_name=scenario_name, # Readable scenario name
        strategy_name=strategy_name, # "First-Fit" or "Best-Fit"
        block_sizes=list(block_sizes), # Starting free-memory block sizes
        process_sizes=list(process_sizes), # Requested process sizes
        events=events, # List of allocation events
        remaining_holes=list(holes), # List of remaining free holes
        total_free_memory=total_free_memory, # Total free memory
        largest_hole=largest_hole, # Largest hole
        total_hole_inspections=total_hole_inspections, # Total hole inspections
        fragmentation_failures=fragmentation_failures, # Fragmentation failures
    )

# ---- end simulate_allocation()

# ======
# Reporting Helpers
# ======

# ---- format_event_table()
def format_event_table(summary: SimulationSummary) -> str:
    """Format a readable per-process allocation table.

    Args:
        summary: Completed simulation summary.

    Returns:
        Multi-line table string.
    """
    # Initialize the table with headers
    lines = [
        "Process  Size  Result                              Inspected",
        "-------  ----  ----------------------------------- ---------",
    ]

    # Iterate through the events
    for event in summary.events:
        # If the process was allocated
        if event.allocated:
            # Format the result text
            result_text = (
                f"addr {event.block_start}, hole {event.block_size_before}, "
                f"left {event.remainder_size}"
            )
        # If the process was not allocated due to external fragmentation
        elif event.fragmentation_blocked:
            result_text = "not allocated (external fragmentation)"
        # If the process was not allocated due to insufficient memory
        else:
            result_text = "not allocated (insufficient memory)"

        # Append the event to the table
        lines.append(
            f"{event.process_label:<7}  "
            f"{event.process_size:>4}  "
            f"{result_text:<36} "
            f"{event.holes_inspected:>9}"
        )

    return "\n".join(lines)

# ---- end format_event_table()

# ---- format_remaining_holes()
def format_remaining_holes(holes: list[MemoryHole]) -> str:
    """Format the remaining holes for console output.

    Args:
        holes: Remaining free holes.

    Returns:
        Human-readable hole description.
    """
    # If there are no remaining holes, return "None"
    if not holes:
        return "None"

    # Format the remaining holes for console output
    return ", ".join(
        f"(start={hole.start}, size={hole.size})"
        for hole in holes
    )

# ---- end format_remaining_holes()

# ---- format_comparison_table()
def format_comparison_table(summaries: list[SimulationSummary]) -> str:
    """Format the scenario comparison summary table.

    Args:
        summaries: Collection of completed simulation summaries.

    Returns:
        Multi-line summary table.
    """
    # Initialize the table with headers
    lines = [
        "Scenario                          Strategy   Alloc  Fail  Free  Largest  Inspections",
        "--------------------------------  ---------  -----  ----  ----  -------  -----------",
    ]

    # Iterate through the summaries
    for summary in summaries:
        lines.append(
            f"{summary.scenario_name[:38]:<38} "
            f"{summary.strategy_name:<9} "
            f"{summary.allocated_count():>5} "
            f"{summary.failed_count():>4} "
            f"{summary.total_free_memory:>4} "
            f"{summary.largest_hole:>7} "
            f"{summary.total_hole_inspections:>11}"
        )

    return "\n".join(lines)

# ---- end format_comparison_table()

# ---- write_csv_report()
def write_csv_report(
    summaries: list[SimulationSummary],
    output_path: Path,
) -> None:
    """Write summary rows to CSV.

    Args:
        summaries: Collection of simulation summaries.
        output_path: Destination CSV path.

    Returns:
        None.
    """
    field_names = [
        "scenario",
        "strategy",
        "allocated",
        "failed",
        "fragmentation_failures",
        "total_free_memory",
        "largest_hole",
        "remaining_hole_count",
        "total_hole_inspections",
        "block_sizes",
        "process_sizes",
        "remaining_hole_sizes",
    ]
    # Open the CSV file
    with output_path.open("w", encoding="utf-8", newline="") as csv_file:
        writer = csv.DictWriter(csv_file, fieldnames=field_names)
        writer.writeheader()
        # Write the summaries to the CSV file
        for summary in summaries:
            writer.writerow(summary.to_csv_row())

# ---- end write_csv_report()

# ======
# Command-Line Wiring
# ======

# ---- parse_args()
def parse_args() -> argparse.Namespace:
    """Parse command-line arguments.

    Args:
        None.

    Returns:
        Parsed argument namespace.
    """
    # Create the parser
    parser = argparse.ArgumentParser(
        description=(
            "Simulate First-Fit memory allocation and optionally compare it "
            "with Best-Fit."
        )
    )

    # Add arguments
    parser.add_argument(
        "--scenario",
        choices=(*BUILT_IN_SCENARIOS.keys(), "all"),
        default="all",
        help="Built-in scenario to run. Default: all.",
    )

    # Add arguments
    parser.add_argument(
        "--mode",
        choices=VALID_MODES,
        default="compare",
        help="Run only First-Fit, only Best-Fit, or both. Default: compare.",
    )

    # Add arguments
    parser.add_argument(
        "--blocks",
        help="Comma-separated custom free-block sizes.",
    )

    # Add arguments
    parser.add_argument(
        "--processes",
        help="Comma-separated custom process sizes.",
    )

    # Add arguments
    parser.add_argument(
        "--csv",
        type=Path,
        help="Optional CSV output path for summary results.",
    )

    # Parse the arguments
    return parser.parse_args()

# ---- end parse_args()
```

Note: The image shows Python code used to scan the free-hole list and place each process under the First-Fit rule, it shows the core logic of First-Fit algorithm.

Figure 2

Python Code Test scenarios and First-Fit vs Best-Fit.

```python
# ============================================================================
# SCENARIO TESTING AND FIRST-FIT / BEST-FIT COMPARISON
# ----------------------------------------------------------------------------
# The code below runs three built-in memory allocation scenarios or a custom set
# of memory blocks and process sizes. It supports compare, first-fit, and
# best-fit modes, allowing First-Fit and Best-Fit to be compared side by side.
# ============================================================================

# ---- run_selected_scenarios()
def run_selected_scenarios(arguments: argparse.Namespace) -> list[SimulationSummary]:
    """Run the requested scenarios and strategies.

    Args:
        arguments: Parsed command-line arguments.

    Returns:
        Completed simulation summaries.

    Raises:
        ValueError: If custom blocks and processes are not both supplied.
    """

    # Check if custom blocks and processes are both supplied
    if bool(arguments.blocks) != bool(arguments.processes):
        raise ValueError(
            "custom runs require both --blocks and --processes"
        )

    # Run custom scenarios if both blocks and processes are supplied
    if arguments.blocks and arguments.processes:
        block_sizes = parse_positive_csv(arguments.blocks)
        process_sizes = parse_positive_csv(arguments.processes)
        scenarios = [
            ScenarioDefinition(
                name="Custom scenario",
                block_sizes=block_sizes,
                process_sizes=process_sizes,
                note="Custom command-line scenario.",
            )
        ]
    # Run built-in scenarios if "all" is specified
    elif arguments.scenario == "all":
        scenarios = list(BUILT_IN_SCENARIOS.values())
    # Run a single built-in scenario
    else:
        scenarios = [BUILT_IN_SCENARIOS[arguments.scenario]]

    # Determine which strategies to run
    strategy_names = (
        ["First-Fit", "Best-Fit"]
        if arguments.mode == "compare"
        else [arguments.mode.title()]
    )

    summaries: list[SimulationSummary] = []

    # Iterate through scenarios and strategies
    for scenario in scenarios:
        for strategy_name in strategy_names:
            # Simulate allocation
            summaries.append(
                simulate_allocation(
                    scenario_name=scenario.name,
                    block_sizes=scenario.block_sizes,
                    process_sizes=scenario.process_sizes,
                    strategy_name=strategy_name,
                )
            )

    return summaries

# ---- end run_selected_scenarios()

# ---- print_report()
def print_report(summaries: list[SimulationSummary]) -> None:
    """Print the detailed scenario report.

    Args:
        summaries: Completed simulation summaries.

    Returns:
        None.
    """
    previous_scenario: str | None = None

    # Iterate through summaries
    for summary in summaries:
        # Print scenario header if it's a new scenario
        if summary.scenario_name != previous_scenario:
            if previous_scenario is not None:
                print()
            print(f"Scenario: {summary.scenario_name}")
            print(f"Memory blocks: {summary.block_sizes}")
            print(f"Process sizes: {summary.process_sizes}")

        # Print strategy header
        print()
        print(f"Strategy: {summary.strategy_name}")

        # Print event table
        print(format_event_table(summary))

        # Print summary
        print(
            "Summary: "
            f"allocated {summary.allocated_count()}/{len(summary.process_sizes)}, "
            f"failed {summary.failed_count()}, "
            f"free memory {summary.total_free_memory}, "
            f"largest hole {summary.largest_hole}, "
            f"remaining holes {len(summary.remaining_holes)}, "
            f"inspections {summary.total_hole_inspections}, "
            f"fragmentation failures {summary.fragmentation_failures}"
        )

        # Print remaining holes
        print(f"Remaining holes: {format_remaining_holes(summary.remaining_holes)}")

        previous_scenario = summary.scenario_name

    # Print comparison summary if more than one summary
    if len(summaries) > 1:
        print()
        print("Comparison Summary")
        print(format_comparison_table(summaries))

# ---- end print_report()


# ---- main()
def main() -> int:
    """Run the command-line program.

    Args:
        None.

    Returns:
        Process exit code.
    """
    arguments = parse_args()
    summaries = run_selected_scenarios(arguments)
    print_report(summaries)

    if arguments.csv:
        write_csv_report(summaries, arguments.csv)
        print()
        print(f"CSV summary written to: {arguments.csv}")

    return 0

# ---- end main()

if __name__ == "__main__":
    raise SystemExit(main())
```

Note: The image shows the Python code used to run multiple test scenarios and compare First-Fit with Best-Fit.

Scenario Testing and Comparison with Best-Fit

Best-Fit uses a different decision rule. Instead of stopping at the first workable hole, it searches all available holes and chooses the smallest hole that can still satisfy the request. The goal is to reduce immediately wasted space inside the selected block and, when possible, preserve larger holes for future allocations. The tradeoff is that Best-Fit usually performs more search work because it must inspect every candidate hole before it can decide. This distinction matters because different request patterns stress the allocator in different ways. A mixed-size workload places more pressure on preserving contiguous space for later large requests, while a clustered small-and-medium workload places more emphasis on search overhead and leftover fragment quality. That pattern appeared clearly in the scenario tests. The simulator was run against three different memory layouts and process-request sequences: a balanced mixed-size case, a clustered small-and-medium case, and a high-fragmentation-pressure case. This testing approach fits the assignment requirement because each run changes the available memory block sizes, the process sizes, or both. See Table 1 for scenario outcomes.

Table 1

First-Fit VS Best-Fit Scenarios Outcomes

Scenario	Strategy	Allocated	Failed	Total Free	Largest Hole	Hole Inspections
Balanced mixed-size requests	First-Fit	3	1	959	300	14
Balanced mixed-size requests	Best-Fit	4	0	533	174	20
Clustered small and medium requests	First-Fit	6	0	550	275	15
Clustered small and medium requests	Best-Fit	6	0	550	300	30
High fragmentation pressure	First-Fit	7	0	300	190	18

High fragmentation pressure	Best-Fit	7	0	300	220	33

Note: The table shows First-Fit and Best-Fit across three memory-allocation scenarios' outputs produced by using the Python code. These measurements have been generated by the Program associated with this paper; see Figure 2 for the different scenarios' Python code.

The balanced scenario is the strongest example of why the algorithms can behave differently. With memory blocks `[100, 500, 200, 300, 600]` and process sizes `[212, 417, 112, 426]`, First-Fit allocated only three of the four processes. The final process failed even though 959 units of memory were still free in total. This was an external-fragmentation failure, because the free memory had been split across holes of sizes 100, 176, 200, 300, and 183, and none of those holes was large enough for the 426-unit request. Best-Fit allocated all four processes in the same scenario because it preserved the 600-unit hole for the later large request. The clustered and pressure scenarios reveal a different pattern. In both cases, First-Fit and Best-Fit allocated every process successfully, but Best-Fit inspected many more holes in order to decide where to place each request. In the clustered scenario, First-Fit completed the run with 15 hole inspections, while Best-Fit required 30. In the pressure scenario, First-Fit required 18 inspections and Best-Fit required 33. This supports the usual practical argument for First-Fit: it often makes an acceptable decision with less search overhead. The simulation also showed that Best-Fit can leave very small fragments behind. In the clustered scenario, Best-Fit left holes of 10 and 5 units, which are much less reusable than the larger leftover holes preserved by First-Fit. That matters because a memory-allocation policy is not judged only by one allocation decision. It is judged by the shape of the memory that remains after many decisions. Esmaeili and Toghraee (2024) explain that concurrent systems depend on careful resource allocation and management across

many simultaneous activities. That broader systems perspective is useful here because memory-allocation quality affects what later work the system can still accept.

When Best-Fit Is Preferable

The Best-Fit algorithm is better suited when allocation success for later medium or large requests matters more than the extra search cost needed to find the smallest adequate hole. The balanced scenario from this project demonstrates that situation clearly. Best-Fit makes sure that a sufficiently large block is allocated for the final 426-unit process, while First-Fit uses the large block earlier and turns its remaining free space into smaller pieces. Best-Fit is also more suited to situations where memory is constrained but CPU time is not. For example, if memory allocation requests arrive at a moderate rate, then the CPU time cost of scanning more holes may be acceptable. This is especially true for server systems that host many differently sized processes over time, because fragmentation can accumulate gradually rather than happen at once.

Another situation where Best-Fit can be better suited is an analytical or batch processing environment where memory allocations and system stability are more important than latency. In other words, First-Fit is best for simple and fast decision making situations, and Best-Fit is best when the system can afford to execute full free hole list scans and the system benefits from preserving larger holes for future jobs. The logical conclusion that can be made is that Best-Fit is not universally better, but it is often better when memory is the main constraint. The clustered scenario showed that the Best-Fit algorithm can create tiny leftover fragments even when it allocates every process's memory needs successfully. Pandikumar et al. (2025) describe memory management or memory allocation as an optimization problem rather than a one-rule-fits-all problem. They argue that fragmentation management and adaptive memory methods for memory allocation need active improvement, especially in systems or workloads that try to balance

efficiency, flexibility, and allocation quality. In other words, the choice between using First-Fit algorithm or the Best-Fit algorithm is a matter of tradeoff and workload requirements.

Conclusion

As shown by the Python program outputs and the Best-Fit comparison, the First-Fit algorithm is easy to implement and effective for many ordinary workloads. The Python scripts illustrated the functionality of the algorithm by allocating each process into the first large enough memory hole, splitting the hole when necessary, and identifying external-fragmentation failures when total free memory existed but no contiguous block was large enough. The comparison with Best-Fits provides the most valuable metrics. It showed that First-Fit usually required fewer hole inspections, which means it made decisions faster and with less search work. However, Best-Fit performed better in the balanced mixed-size scenario because it preserved a larger block for a later large request and therefore allocated all four processes successfully. This shows that First-Fit is best when simplicity and speed matter most, while Best-Fit is best when memory is tight, process sizes vary significantly, and allocation success is more important for process future memory requests than search overhead.

References

Bazuku, R., Anab, A., Gyemerah, S., & Daabo, M. I. (2023). An overview of computer operating systems and emerging trends. *Asian Journal of Research in Computer Science, 16*(4), 161-177. https://doi.org/10.9734/AJRCOS/2023/v16i4380

CSU Global. (n.d.). *Lecture 4: Memory management* [Interactive lecture]. CSC507: Foundation of Operating Systems. Colorado State University Global.

Esmaeili, M., & Toghraee, M. (2024). Understanding concurrent processing: Challenges in operating systems. *Journal of Computer Based Parallel Programming, 9*(3), 18-23.

Pandikumar, S., Jakaraddi, H. R., & Sevugapandi, N. (2025). Chapter 7: Impact of AI in the design of operating systems: An overview. In N. Revathy & S. Pandikumar (Eds.), *Shaping the digital future: From algorithms to intelligence (pp. 64-71).* QTanalytics® India.

Wilson, P. R., Johnstone, M. S., Neely, M., & Boles, D. (1995). Dynamic storage allocation: A survey and critical review. *Lecture Notes in Computer Science, 1–116*. https://doi.org/10.1007/3-540-60368-9_1